

**Competition Topic: Promotion of innovation in the semiconductor industry to bring about technological progress and industry upgrade**

**Competition Category: Device Design and Development**

**Team ID: 2526\_C\_0061**

**Supervisor: Prof. Arun Tej Mallajosyula**

**Team Captain: Siddarth Gupta**

**Team Members: Siddarth Gupta, Varunan Balasubramanian, Lalitya Marathe, Ananya Dongsarwar**

# 1. Project Title

Reduction of Energy Usage and Effective Area by Statistical Precision Scaling and Analog Layer Cascading in Analog Compute-in-Memory.

## 2. Problem Identification

Analog Compute-in-Memory (CIM) architectures aim to improve energy efficiency by performing multiply-accumulate (MAC) operations directly inside memory arrays. In an ideal crossbar array, a neural network layer computes

$$I_j = \sum_i G_{ij} V_i$$

where  $G_{ij}$  is the conductance representing a stored weight,  $V_i$  is the applied input voltage, and  $I_j$  is the resulting output current. This physical implementation eliminates digital multipliers and significantly reduces data movement energy. As a result, the crossbar computation itself is highly energy efficient. However, the output of each layer is an analog current and must be converted to digital form using an Analog-to-Digital Converter (ADC) before activation and forwarding to the next layer. In practical systems, this conversion dominates overall energy consumption. Empirically,

$$E_{ADC} \gg E_{crossbar}$$

where  $E_{crossbar}$  is the energy consumed by analog MAC computation and  $E_{ADC}$  is the energy consumed by digitization. For a network with  $L$  layers,

$$E_{ADC,total} = L \cdot E_{ADC}.$$

Thus, total system energy scales with the number of ADC activations rather than with computation cost. At the same time, keeping signals in the analog domain introduces accumulated noise due to device variability and circuit non-idealities. The central problem addressed in this work is reducing the number of ADC conversions in multi-layer analog CIM architectures while maintaining effective computational precision.

## 3. Introduction to our solution

To address ADC energy dominance, we propose a statistically motivated architectural modification that enables reduced digitization without sacrificing precision. Our approach consists of two ideas:

1. Reduce intrinsic device-level variability using conductance splitting.
2. Use the resulting precision margin to cascade multiple layers in the analog domain before digitization.

### 3.1 Statistical Reduction of Variability

In a conventional crossbar, each weight is stored in a single flash transistor:

$$G = G_0 + \delta G$$

where  $G_0$  is the intended conductance and  $\delta G$  represents programming noise. Instead of storing the weight in one device, we split it into  $n$  parallel transistors:

$$G_k = \frac{G_0}{n} + \delta G_k$$

with independent  $\delta G_k$  of standard deviation  $\sigma_{cell}$ . The effective conductance becomes

$$G_{eff} = \sum_{k=1}^n G_k = G_0 + \sum_{k=1}^n \delta G_k.$$

Since independent variances add,

$$Var(G_{eff}) = n\sigma_{cell}^2,$$

thus

$$\sigma_{eff} = \sqrt{n}\sigma_{cell}.$$

For a single large device,

$$\sigma_{single} \approx n\sigma_{cell}.$$

Therefore,

$$\sigma_{eff} = \frac{\sigma_{single}}{\sqrt{n}}.$$

Splitting into  $n$  devices reduces standard deviation by  $\sqrt{n}$  and reduces noise power by  $n$ .

#### Case of Four-Way Splitting

For  $n = 4$ ,

$$\sigma_{eff} = \frac{\sigma_{single}}{2}.$$

Noise power reduces by 4×, increasing SNR by 4×. Since

$$10 \log_{10}(4) \approx 6 \text{ dB},$$

and ideal ADCs gain  $\approx 6$  dB per bit, four-way splitting provides nearly one additional effective bit of intrinsic precision. This precision margin is used to tolerate analog cascading error.

### 3.2 Reduction of ADC Energy and Area

In conventional CIM, computation of the first layer happens through the first multiply accumulate module and the output is converted to digital domain for the activation function to be executed and then its converted back to analog domain for the next

layer of multiply accumulation. While not as significant, due to them already being quite power efficient we also effectively halve the number of DACs used. For  $L$  layers,

$$E_{ADC,total} = L \cdot E_{ADC}.$$

Since  $E_{ADC} > E_{crossbar}$ , energy is ADC-dominated. We instead cascade two layers, so now data flows directly from our first layer into our analog activation function and directly goes to the next layer without any conversions. This halves ADC activations:

$$E_{ADC,total,new} = \frac{L}{2} \cdot E_{ADC}.$$

ADC resolution remains unchanged; energy reduction comes from fewer conversions.

### 3.3 Analog Implementation of Activation Function

To enable full analog cascading, the activation function must also be implemented in the analog current domain. In our architecture, the multiply-accumulate (MAC) module produces a differential current output consisting of two branches:

$$I^+ \quad \text{and} \quad I^-$$

representing the positive and negative weighted sums, respectively.

#### Current Subtraction

Both differential currents are first passed through dedicated op-amp based current buffers to isolate the crossbar array from loading effects. The buffered positive current  $I^+$  is directly injected into a common summing node. The negative current  $I^-$  is mirrored using an NMOS current mirror, and this mirrored current is sunk from the same node. At the summing node, the net current becomes

$$I_{node} = I^+ - I^-.$$

Thus, differential subtraction is achieved entirely in the current domain.

#### ReLU Operation

The resulting current  $I_{node}$  is then fed into a second current mirror stage designed such that it only mirrors positive currents. If  $I_{node} > 0$ , the mirror copies the current to the output. If  $I_{node} \leq 0$ , no current flows through the mirror, effectively clamping the output to zero. Therefore, the activation function implemented is

$$I_{out} = \begin{cases} I^+ - I^-, & I^+ > I^- \\ 0, & I^+ \leq I^- \end{cases}$$

This realizes the Rectified Linear Unit (ReLU) activation entirely in the analog domain without requiring digitization.

### 3.4 Cascading of Two Analog Layers

When cascading two analog layers, three primary error sources must be considered:

- Crossbar variability from Layer 1 ( $\epsilon_1$ ),
- Error introduced by the activation circuit ( $\epsilon_{act}$ ),
- Crossbar variability from Layer 2 ( $\epsilon_2$ ).

The output of the first layer is

$$y_1 = W_1 x + \epsilon_1.$$

After the current-subtraction and ReLU circuit, additional error arises due to finite op-amp gain, current mirror mismatch, threshold mismatch, and finite output resistance. The signal becomes

$$\tilde{y}_1 = y_1 + \epsilon_{act}.$$

The second layer then produces

$$y_2 = W_2 \tilde{y}_1 + \epsilon_2.$$

Assuming independent noise sources with variances  $\sigma_1^2$ ,  $\sigma_{act}^2$ , and  $\sigma_2^2$ , the total variance is

$$\sigma_{total}^2 = \sigma_1^2 + \sigma_{act}^2 + \sigma_2^2.$$

After four-way conductance splitting,

$$\sigma_1 = \sigma_2 = \frac{\sigma_0}{2}.$$

Thus,

$$\sigma_{total}^2 = \frac{\sigma_0^2}{2} + \sigma_{act}^2.$$

To ensure that the cascaded two-layer system does not exceed the original single-layer variability  $\sigma_0$ , we require

$$\sigma_{total} \leq \sigma_0.$$

Substituting,

$$\frac{\sigma_0^2}{2} + \sigma_{act}^2 \leq \sigma_0^2.$$

Rearranging,

$$\sigma_{act}^2 \leq \frac{\sigma_0^2}{2}.$$

Taking square root,

$$\sigma_{act} \leq \frac{\sigma_0}{\sqrt{2}}.$$

Since  $\frac{1}{\sqrt{2}} \approx 0.707$ , the activation circuit standard deviation must be less than approximately 70% of the original crossbar variability. In practical implementations, flash programming variability is typically on the order of 5–10%, while well-designed current mirrors and op-amp buffers can achieve mismatch below 1–2%. Therefore,

$$\sigma_{act} \ll \frac{\sigma_0}{\sqrt{2}},$$

and the activation circuit does not dominate total error. so we conclude that four-way conductance splitting provides nearly one additional effective bit of intrinsic precision, which compensates for both crossbar variability and activation circuit non-idealities. This enables reliable two-layer analog cascading while reducing ADC usage by half.

## 4. Experimental Design

The proposed architecture was implemented and validated at the circuit level using 180 nm CMOS technology. All modules were designed in current mode to preserve signal integrity and minimize unnecessary voltage-domain conversions.

### 4.1 MAC Crossbar Module

The MAC module implements differential current-mode multiplication using conductance-weighted branches. Each logical weight is represented by multiple parallel flash devices to achieve statistical averaging and reduced standard deviation.

Input voltages are applied across conductance elements, producing weighted currents according to Ohm's law. These currents are summed at a common node to realize the dot-product operation.

Key characteristics evaluated:

- Linearity of current summation,
- Sensitivity to conductance variation,
- Current range and dynamic headroom,
- Static and dynamic power consumption.

### 4.2 Current Subtraction and Analog ReLU Module

The differential outputs of the MAC module are processed using two op-amp-based current buffers to isolate loading effects. The positive branch is directly forwarded to the subtraction node, while the negative branch is mirrored using an NMOS current mirror and subtracted from the same node.

The resulting current represents the differential subtraction:

$$I_{out} = I^+ - I^-.$$

To implement the ReLU activation function, a subsequent current mirror structure selectively mirrors only positive currents while blocking negative currents. This ensures:

$$I_{ReLU} = \begin{cases} I_{out}, & I_{out} > 0 \\ 0, & I_{out} \leq 0. \end{cases}$$

## 5.Simulation Results

To address ADC energy dominance, we propose a statistically motivated architectural modification that enables reduced digitization without sacrificing precision.

### 5.1 Power Analysis: ADC-Free Advantage

We compare one  $3 \times 3$  differential MAC layer between:

- Proposed ADC-free architecture
- Conventional analog CIM architecture (with ADCs)

#### Proposed Architecture

Measured total power for one MAC layer:

$$P_{\text{ours}} = 112.134 \mu W \quad (1)$$

This includes full differential MAC computation. No ADC is required.

#### Conventional Architecture

For a  $3 \times 3$  differential MAC:

- 3 columns
- 1 differential ADC per column
- $N = 3$

As a safe approximation, we assume:

$$P_{\text{ADC}} = 80 \mu W \quad \text{per ADC} \quad (2)$$

Total ADC power becomes  $80 \times$  the number of ADCs used, we currently use 1 ADC per column, so that's 3 ADCs in total so 240uW of power in total.

#### Power Reduction Achieved

Absolute reduction:

$$\Delta P = 240 - 112.134 \quad (3)$$

$$\Delta P = 127.866 \mu W \quad (4)$$

Percentage reduction:

$$\frac{127.866}{240} \times 100 = 53.3\% \quad (5)$$

## Key Result

The proposed ADC-free architecture achieves:

### **53.3% lower power per 3×3 MAC layer**

so as compared to a conventional MAC layer our proposed MAC layer has a 53.3% reduction in power, since ADCs use up almost 85% of power in a conventional analog compute in memory chip, we will approximately decrease about 40% of power in the overall power.

## 5.2 Statistical Error Analysis

The measured statistical variation of the proposed conductance-splitting implementation resulted in a standard deviation of 3.1 % relative to the nominal programmed conductance value. This aligns with the analytical expectation derived in Section 2, where splitting a single logical weight into four parallel devices reduces variability by a factor of  $\sqrt{n}$ , yielding a twofold reduction in standard deviation for  $n=4$ . The obtained 3.1 % deviation confirms that the redundancy-based statistical averaging performs as intended and remains within the precision margin required for reliable two-layer analog cascading. This validates that the implemented variability mitigation strategy achieves the target stability discussed earlier.



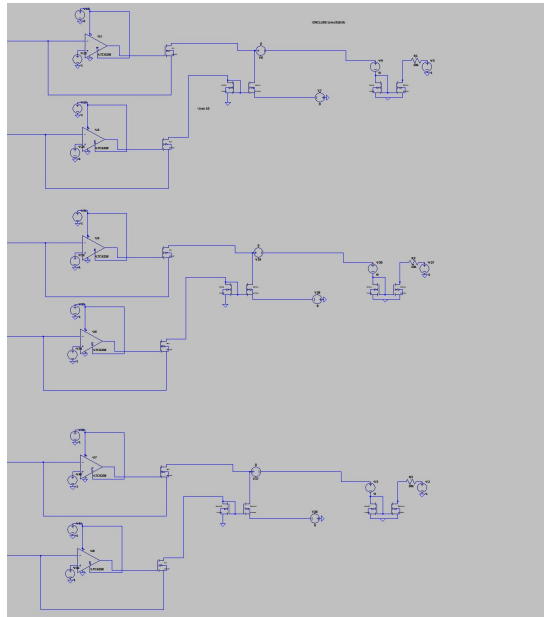
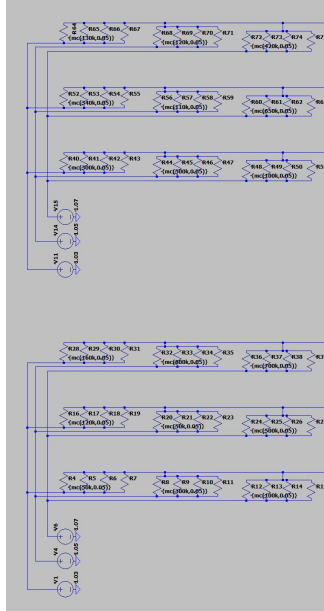
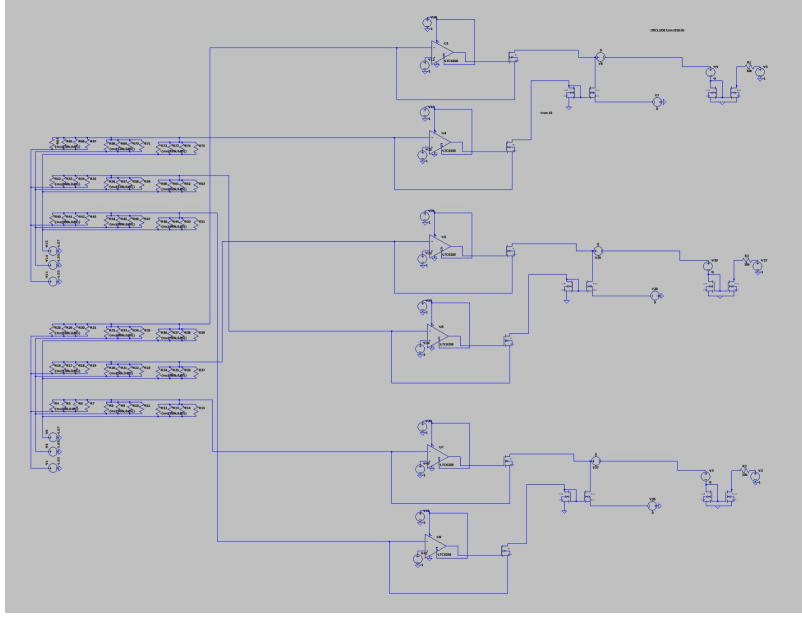


Figure 1: Circuit-level implementation of the proposed analog cascading architecture. Top: Overall architecture. Middle: MAC module. Bottom: Subtractor + ReLU module.

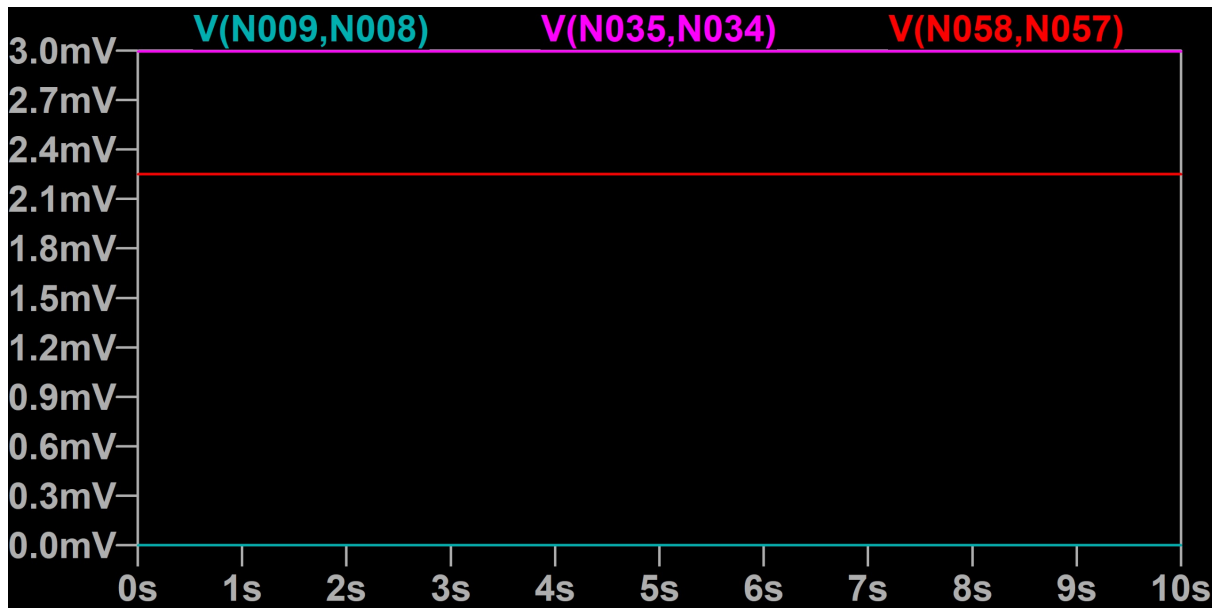


Figure 2: Simulation of the circuit for a given input vector : 3mV,5mV,7mV and weights(in terms of resistance, actual weight is  $1/R = G$ )  $I^-[(32.5k, 30k, 105k), (85k, 27.5k, 162.5k), (75k, 125k, 25k)]$ ,  $I^+[(40k, 200k, 175k), (30k, 12.5k, 125k), (12.5k, 75k, 25k)]$ , top line(pink, 3mV), middle line(red, 2.2mV), bottom line(aqua,0V) are the 3 outputs

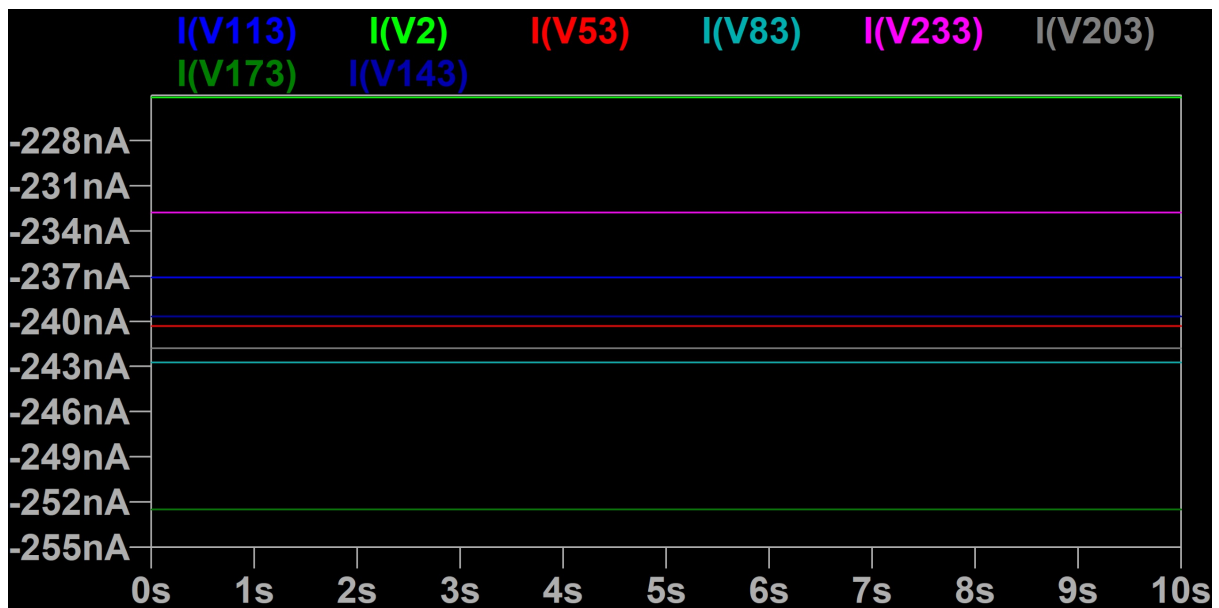


Figure 3: Final Output current distribution across multiple Monte Carlo runs

## 6. Project Summary and Future Work

### 6.1 Project Summary

This work addresses the primary energy bottleneck in analog Compute-in-Memory (CIM) architectures: repeated analog-to-digital conversions. Although crossbar arrays perform multiply-accumulate (MAC) operations efficiently using device physics, overall system energy is dominated by ADC usage, where

$$E_{ADC} > E_{crossbar}.$$

To mitigate this, we introduced a statistically motivated architectural modification consisting of:

- Splitting each weight into  $n$  parallel flash devices to reduce intrinsic variability by a factor of  $\sqrt{n}$ .
- Implementing four-way splitting to obtain nearly one additional effective bit of precision.
- Designing a fully analog current-mode ReLU activation function using differential current subtraction and selective current mirroring.
- Cascading two neural network layers in the analog domain before digitization.

By using the precision margin gained from conductance splitting, the accumulated noise from crossbar variability and activation circuit non-idealities remains below the original single-layer variability. As a result, ADC activations are reduced by approximately 50% without lowering resolution, leading to significant energy and area savings.

### 6.2 Future Work

Several extensions can further enhance the proposed architecture.

#### (1) Bit Slicing for Higher Precision

Bit slicing is a technique in which a multi-bit weight is implemented using multiple lower-resolution slices distributed across parallel crossbar arrays. Instead of storing a full high-precision weight in a single conductance value, each bit (or group of bits) is stored in separate arrays and weighted appropriately during accumulation. For example, an 8-bit weight can be decomposed into four 2-bit slices. The outputs of these slices are scaled and summed to reconstruct the effective high-precision result. Bit slicing provides the following advantages:

- Improved effective precision,
- Reduced sensitivity to device non-linearity,
- More robust handling of analog variability.

Integrating bit slicing with statistical conductance splitting can further increase inference accuracy while preserving analog efficiency.

## **(2) Technology Scaling Beyond 180 nm**

The current circuit implementation targets 180 nm technology. Scaling to lower technology nodes (e.g., 65 nm or 28 nm) offers:

- Reduced parasitic capacitances,
- Lower dynamic switching energy,
- Higher integration density,
- Smaller current mirror mismatch due to improved layout control.

Technology scaling would directly reduce ADC area and energy while allowing more compact analog cascading blocks.

## **(3) Eliminating Op-Amps in Current Subtraction**

The present design uses op-amp-based current buffers for stability and isolation. Future designs can eliminate operational amplifiers entirely by:

- Using purely transistor-level current steering,
- Employing differential pair subtraction,
- Designing passive current-mode virtual ground nodes.

Removing op-amps would significantly reduce power consumption, area, and design complexity, enabling a fully transistor-level analog accelerator.

## **(4) Data-Driven Weight Pruning and Sparsity Exploitation**

From a data science perspective, many trained neural networks contain redundant or near-zero weights. By applying pruning techniques and structured sparsity optimization:

- Redundant weights can be removed,
- Effective crossbar size can be reduced,
- Computational load decreases,
- ADC usage per inference reduces further.

Combining hardware-level efficiency with model-level sparsity can significantly enhance overall system performance.

## **(5) Deeper Analog Cascading**

With improved device precision and optimized activation circuits, cascading more than two layers in the analog domain may become feasible. A generalized analysis

of  $n$ -way splitting and  $m$ -layer cascading can determine the optimal trade-off between precision and ADC reduction.

**Overall Outlook:** Modern neural networks have been shown to operate effectively under reduced numerical precision, often requiring only 6–8 bits for accurate inference. Therefore, this work does not aim to aggressively increase precision beyond practical requirements. Instead, the available intrinsic precision provided by statistical conductance splitting is strategically utilized to enable analog layer cascading and reduce digitization overhead.

By prioritizing reductions in ADC usage, silicon area, and overall power consumption rather than pursuing unnecessarily high bit resolution, the proposed architecture aligns with the practical precision demands of real-world neural network inference workloads. This power-first design philosophy enables scalable and energy-efficient analog Compute-in-Memory systems suitable for edge and embedded AI applications.

## 7. References

1. A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, *Accelerating AI Using Next-Generation Hardware: Possibilities and Challenges with Analog In-Memory Computing*, Nature Electronics, vol. 3, no. 4, pp. 230–240, 2020.
2. A. Shafiee et al., *ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars*, Proceedings of the 43rd International Symposium on Computer Architecture (ISCA), 2016, pp. 14–26.
3. W. Zhang, R. Khaddam-Aljameh, M. Le Gallo, A. Sebastian, and E. Eleftheriou, *Precision Modeling and Evaluation of Analog In-Memory Computing Systems*, IEEE Transactions on Circuits and Systems I, vol. 66, no. 9, pp. 3538–3551, 2019.
4. P. Chi et al., *PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory*, ISCA, 2016.
5. S. Yu, *Neuro-Inspired Computing with Emerging Nonvolatile Memory Technologies*, Proceedings of the IEEE, vol. 106, no. 2, pp. 260–285, 2018.
6. M. Hu et al., *Dot-Product Engine for Neuromorphic Computing: Programming 1T1M Crossbar to Accelerate Matrix-Vector Multiplication*, Design Automation Conference (DAC), 2016.
7. H. Tanaka et al., *Spread Programming Scheme for NAND Flash Memory*, IEEE Transactions on Electron Devices, vol. 58, no. 7, pp. 1945–1951, 2011.
8. S. Han, J. Pool, J. Tran, and W. Dally, *Learning Both Weights and Connections for Efficient Neural Networks*, Advances in Neural Information Processing Systems (NeurIPS), 2015.
9. B. Jacob et al., *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018.